

LD3 Sparse Framework Density Specification

Framework-vertex occupancy, framework-edge arc length, adaptive quadrature, provenance,
planning, and rendering

mdstats development specification

2026-07-20

Contents

1	Status and scope	2
2	Scientific measures	2
2.1	Framework-vertex occupancy	2
2.2	Framework-edge arc length	2
3	Inputs and public options	3
4	Edge quadrature	3
4.1	Base resolution policy	3
4.2	Automatic mode	3
4.3	Explicit mode	4
4.4	Midpoint rule and exact weights	4
5	Resolved quadrature record	4
6	Sparse source batches and provenance	5
7	Sparse numerical path	5
8	Transactional planning	5
9	Field metadata	6
10	Rendering	6
11	Failure semantics	6
12	Acceptance criteria	7
12.1	Scalar-field equivalence	7
12.2	Measure and symmetry	7
12.3	Quadrature convergence	7
12.4	Rendering	7
12.5	Compatibility	7
13	Required focused tests	8
14	Completion condition	8
15	References	8

1 Status and scope

This specification governs architecture gate **LD3** for `mdstats`. It begins from `mdstats 0.19.48a0`, where block-sparse atomic density fields already support exact canonical convolution, highest-density-region thresholds, logical-node clouds, and periodic triangular meshes.

LD3 extends the production `local_sparse` backend to the two framework measures:

1. projected framework-vertex occupancy;
2. framework-edge arc length, using either projected framework edges or retained atom-resolved path segments.

LD3 also makes framework edge quadrature resolution-aware and integrates both channels with transactional scene planning and the existing LD2 renderer.

LD3 does **not** implement automatic backend selection, multilevel refinement, variable-bandwidth kernel density estimation, GPU kernels, or mesh decimation.

2 Scientific measures

2.1 Framework-vertex occupancy

Let V_t be the retained projected framework vertices in frame t , with normalized frame weights w_t . The occupancy measure is

$$\mu_V = \sum_t w_t \sum_{v \in V_t} \delta_{\mathbf{x}_{t,v}}.$$

Its total measure is the retained projected vertex count,

$$M_V = \sum_t w_t |V_t| = |V|,$$

and its scalar density has units

$$[\rho_V] = \text{\AA}^{-3}.$$

2.2 Framework-edge arc length

For retained segment e in frame t , let $\gamma_{t,e}(s)$ be its straight Cartesian segment parameterization and $L_{t,e}$ its length. The edge measure is

$$\mu_E(A) = \sum_t w_t \sum_e \int_0^{L_{t,e}} \mathbf{1}_A(\gamma_{t,e}(s)) ds.$$

Its total measure is

$$M_E = \sum_t w_t \sum_e L_{t,e},$$

and its scalar density has units

$$[\rho_E] = \text{\AA}^{-2}.$$

Occupancy and arc length are never merged into one channel or assigned one common unit.

3 Inputs and public options

The stage extends `FrameworkDensityOptions` with the following edge-quadrature controls:

```
FrameworkDensityOptions(  
    edge_source="projected",           # projected / atomic_paths  
    edge_sample_spacing=0.20,         # angstrom  
    edge_sample_spacing_mode="auto",  # auto / explicit  
    edge_quadrature_refinement_levels=2, # integer in [0, 8]  
    storage_options=DensityStorageOptions(  
        grid_backend="local_sparse",  
        local_block_shape=(16, 16, 16),  
    ),  
    kernel_options=DensityKernelOptions(  
        smoothing_operator="discrete_periodized_v1",  
    ),  
)
```

For `grid_backend="local_sparse"`, `discrete_periodized_v1` is mandatory. `legacy_spectral_v1` remains supported only by the dense backend.

`edge_source="projected"` uses the projected framework edges. The `"atomic_paths"` mode uses every retained path segment in the atom-resolved framework view. Both modes use the same density estimator and normalization.

4 Edge quadrature

4.1 Base resolution policy

After the density grid and Gaussian bandwidth are resolved, define the three Cartesian logical-grid basis vectors

$$\mathbf{b}_i = \frac{\mathbf{a}_i}{N_i},$$

and

$$h_{\text{axis},\min} = \min_i \|\mathbf{b}_i\|_2.$$

For $\sigma > 0$, the base quadrature interval is

$$h_E^{(0)} = \min \left(h_{\text{axis},\min}, \frac{\sigma}{2} \right).$$

For the zero-bandwidth identity path,

$$h_E^{(0)} = h_{\text{axis},\min}.$$

4.2 Automatic mode

The deterministic, transactionally predictable automatic interval is

$$h_E = \min \left(h_{E,\text{nominal}}, \frac{h_E^{(0)}}{2^r} \right),$$

where r is `edge_quadrature_refinement_levels`. The default is $r = 2$. Allowing $r = 0$ exposes the unrefined base policy without changing source code.

The refinement depth is fixed before Phase-B exact planning. The implementation does not construct trial scalar fields before global scene approval. The default $r = 2$ is certified by the focused convergence suite described below; it is a project policy, not a universal mathematical error estimator.

4.3 Explicit mode

With `edge_sample_spacing_mode="explicit"`,

$$h_E = h_{E,\text{nominal}}.$$

The supplied interval is authoritative. If it is coarser than $h_E^{(0)}$, the implementation emits a diagnostic warning and records `edge_sample_spacing_underresolved=True`. It never silently refines, coarsens, or changes an explicit interval.

4.4 Midpoint rule and exact weights

For a segment of length L , use

$$n = \max\left(1, \left\lceil \frac{L}{h_E} \right\rceil\right)$$

midpoint samples. Ratios within 10^{-12} relative tolerance of an integer use the nearest integer to avoid platform-dependent extra samples.

The samples are

$$\mathbf{x}_j = \mathbf{x}_0 + \frac{j + 1/2}{n}(\mathbf{x}_1 - \mathbf{x}_0), \quad j = 0, \dots, n-1,$$

with weights

$$q_j = w_t \frac{L}{n}.$$

The final weight on each segment is corrected by the floating-point residual so that

$$\sum_{j=0}^{n-1} q_j = w_t L$$

exactly to the stored precision. A final global residual correction preserves M_E .

The local-sparse path canonicalizes segment orientation by Cartesian endpoint order before sampling. Reversing every segment endpoint therefore leaves sparse sample positions, weights, and the final scalar field unchanged. The dense compatibility path retains historical input-segment ordering; its mathematical measure is orientation invariant, subject only to floating-point accumulation order.

5 Resolved quadrature record

The public immutable record is:

```
@dataclass(frozen=True, slots=True)
class FrameworkEdgeQuadratureResolution:
    mode: Literal["auto", "explicit"]
    nominal_spacing: float
    realized_spacing: float
    axis_min_spacing: float
```

```

gaussian_half_spacing: float | None
policy_spacing: float
refinement_levels: int
explicit_underresolved: bool

```

The same resolver is called by Phase-B planning and Phase-C realization. A mismatch between planned and realized sample counts is an error.

6 Sparse source batches and provenance

Both channels reduce to `PeriodicWeightedSamples3D`.

For vertices:

```

measure_kind      occupancy
measure_units     count
physical_units    angstrom^-3
source_kind       framework_vertex_occupancy

```

For edges:

```

measure_kind      arc_length
measure_units     angstrom
physical_units    angstrom^-2
source_kind       framework_edge_arc_length

```

Persistent provenance records atom indices and canonical framework vertex or edge keys. Transient `sample_group_ids` associate quadrature samples with their retained edge/path segment without storing one full source key per sample.

Source keys are canonical, JSON-safe tagged tuples. Framework graph keys are encoded with their canonical `to_dict()` payload and deterministic key ordering.

7 Sparse numerical path

For either channel, the production path is

```

weighted periodic samples
-> deterministic periodic CIC aggregation
-> canonical sparse stencil support
-> deterministic stencil-major accumulation
-> exact total-measure normalization
-> deterministic block packing
-> PeriodicBlockScalarField3D

```

The estimator, node convention, kernel normalization, and HDR semantics are identical to LD1-A/LD1-B. Framework channels do not introduce a second sparse estimator.

The framework-vertex resolution reference remains the valid framework-vertex positional spread. Edge quadrature samples have their own CIC covariance diagnostic, but do not redefine the adaptive-resolution reference.

8 Transactional planning

Phase A computes conservative source bounds before sample arrays are built. For auto edge quadrature, the bound is limited by `max_density_samples` because the final grid and bandwidth are not yet known.

Phase B resolves density numerics and edge quadrature, constructs the exact weighted sample batches, and computes:

- occupied CIC nodes;
- canonical stencil support;

- target sparse nodes;
- active block indices and valid masks;
- nonzero-node, stored-slot, block, kernel-pair, and planning-byte counts.

All requested atomic and framework channels are globally approved together before Phase C allocates scalar block values. Phase C recomputes the same source batch and asserts that realized counts do not exceed the approved plan.

No limit failure may silently increase h_E , h , or σ , remove segments, or change the edge source.

9 Field metadata

Every framework field records at least:

field and schema identifiers
 storage backend and smoothing operator
 physical and measure units
 grid shape and realized intervals
 Gaussian bandwidth and broadening diagnostics
 resolution reference source
 structured source provenance
 approved Phase-B plan and scene approval ID
 realized block/storage counts

Edge fields additionally record:

edge source
 spacing mode
 nominal and realized spacing
 base grid/kernel policy spacing
 refinement levels
 under-resolution flag
 quadrature sample count
 quadrature weight sum
 total retained mean edge length

10 Rendering

The existing LD2 services consume the backend-neutral scalar-field and node-access contracts. No framework-specific mesh or cloud implementation is introduced.

Both framework channels support:

- positive-mass HDR thresholds;
- logical-node clouds without dense materialization;
- canonical sparse meshes;
- expanded-cell `match_graph` replication;
- periodic seam validation and winding fallback;
- independent legend toggles and trace provenance.

Vertex and edge channels retain separate field keys, units, labels, and trace groups.

11 Failure semantics

GraphStyleError invalid source, spacing mode, interval, refinement depth, kernel/backend pairing, or shared density option.

GraphAdapterError invalid segment shapes, misaligned frame weights, empty nondegenerate edge measure, unstable projected/path identity, or unserializable source provenance.

GraphComplexityError sample, CIC, stencil, kernel-pair, block, planning-memory, field-memory, mesh, or scene-wide resource limit exceeded.

Failures are explicit. The implementation does not drop edges, change the selected source, reduce shell fractions, or coarsen the field.

12 Acceptance criteria

12.1 Scalar-field equivalence

For matched canonical dense and sparse fields:

```
relative L1 field error          <= 2e-12
relative Linfinity error         <= 5e-12
absolute integral error          <= 5e-13 * max(1, total_measure)
HDR threshold difference         <= 5e-12 relative
HDR achieved-mass difference     <= 5e-12 absolute
```

12.2 Measure and symmetry

- Vertex integral equals retained projected vertex count.
- Edge integral equals retained weighted total edge/path length.
- Segment orientation reversal leaves field values and measure unchanged exactly.
- Integer periodic translations leave both fields unchanged.
- Projected and atom-resolved path modes are both supported.
- Vertex and edge units remain $\text{\AA}^3$ and $\text{\AA}^2$.

12.3 Quadrature convergence

For the automatic default interval and a comparison field with half that interval:

```
relative L1 field difference      <= 2e-3
relative Linfinity field difference <= 1e-2
maximum relative HDR threshold diff <= 1e-3
```

The certification suite includes projected and atom-resolved paths, orthogonal and skewed cells, and periodic boundary crossings. These tests validate the default refinement policy; they do not claim a universal a posteriori error bound.

12.4 Rendering

For nonwinding sparse framework shells:

```
interior edge-incidence failures = 0
unpaired periodic boundary edges = 0
maximum boundary seam mismatch <= 1e-10 * L_ref
duplicate final faces = 0
degenerate final faces = 0
```

Cloud and mesh rendering must dispatch through the common renderer without framework-channel special cases.

12.5 Compatibility

- The default dense atomic and framework-vertex scientific fields remain unchanged.
- Dense framework-edge reproduction of the pre-LD3 estimator is available with `edge_sample_spacing_mode="explicit"` and the previous spacing.
- The new default automatic edge policy is an intentional, versioned scientific migration and is recorded in metadata and the release notes.

13 Required focused tests

1. dense-versus-sparse framework-vertex equivalence;
2. dense-versus-sparse projected-edge equivalence;
3. dense-versus-sparse atom-resolved-path equivalence;
4. exact vertex and edge normalization;
5. orientation reversal;
6. periodic face/edge/corner crossings in a skewed cell;
7. deterministic block ordering and serialization;
8. structured vertex and edge provenance;
9. explicit under-resolved spacing warning;
10. automatic spacing and refinement-depth resolution;
11. quadrature convergence against half spacing;
12. scene planning and realization-count agreement;
13. mesh and node-cloud rendering for both channels;
14. failure before block allocation for every hard limit;
15. dense legacy-compatibility fixture in explicit-spacing mode.

14 Completion condition

LD3 is complete when both framework channels are production-capable under `grid_backend="local_sparse"`, satisfy the scientific and resource gates above, and render through the LD2 backend-neutral services without dense materialization.

15 References

1. R. W. Hockney and J. W. Eastwood, *Computer Simulation Using Particles*, Adam Hilger, 1988. Reprint DOI: 10.1201/9780367806934.
2. R. J. Hyndman, “Computing and Graphing Highest Density Regions,” *The American Statistician* **50** (1996), 120-126. DOI: 10.1080/00031305.1996.10474359.
3. W. E. Lorensen and H. E. Cline, “Marching Cubes: A High Resolution 3D Surface Construction Algorithm,” *Computer Graphics* **21** (1987), 163-169. DOI: 10.1145/37402.37422.
4. T. Lewiner, H. Lopes, A. W. Vieira, and G. Tavares, “Efficient Implementation of Marching Cubes’ Cases with Topological Guarantees,” *Journal of Graphics Tools* **8** (2003), 1-15. DOI: 10.1080/10867651.2003.10487582.